EXPERIMENT – 1

AIM: Implement a Multilayer Perceptron Algorithm for MNIST Hand written digit classification.

A multilayer perceptron (MLP) Neural network belongs to the feedforward neural network. It is an Artificial Neural Network in which all nodes are interconnected with nodes of different layers.

Frank Rosenblatt first defined the word Perceptron in his perceptron program. **Perceptron is a basic unit of an artificial neural network that defines the artificial neuron in the neural network.** It is a supervised learning algorithm containing nodes' values, activation functions, inputs, and weights to calculate the output.

The Multilayer Perceptron (MLP) Neural Network works only in the forward direction. All nodes are fully connected to the network. Each node passes its value to the coming node only in the forward direction. The **MLP neural network uses a Backpropagation algorithm to increase the accuracy of the training model.**

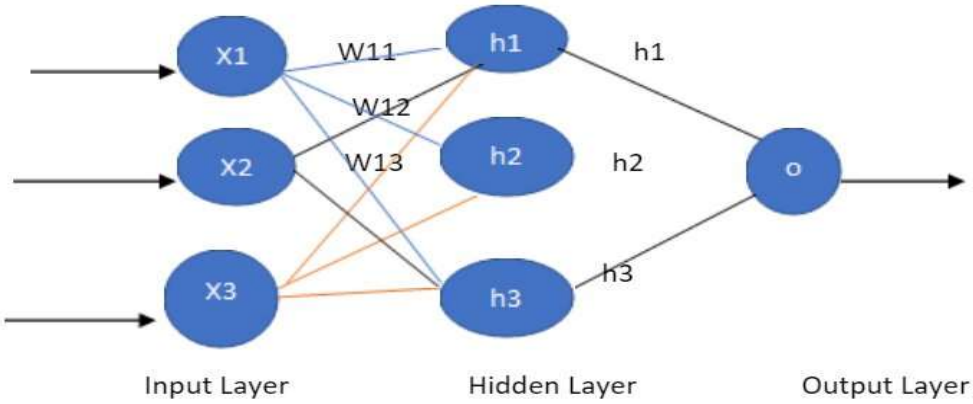


Diagram Of MultiLayer Perceptron Neural Network

Working of Multilayer Perceptron Neural Network

- The input node represents the feature of the dataset.
- Each input node passes the vector input value to the hidden layer.
- In the hidden layer, each edge has some weight multiplied by the input variable. All the production values from the hidden nodes are summed together. To generate the output
- The activation function is used in the hidden layer to identify the active nodes.
- The output is passed to the output layer.
- Calculate the difference between predicted and actual output at the output layer.
- The model uses backpropagation after calculating the predicted output.

Back Propagation Algorithm

The backpropagation algorithm is used in a Multilayer perceptron neural network **to increase the accuracy of the output by reducing the error in predicted output and actual output.**

Keras

Keras is a high-level, deep learning API developed by Google for implementing neural networks. It is written in Python and is used to make the implementation of neural networks easy. It also supports multiple backend neural network computation.

Keras is an open-source deep-learning framework that gained attention due to its user-friendly interface.

Keras offers ease of use, flexibility, and the ability to run seamlessly on top of TensorFlow.

TensorFlow

TensorFlow is a popular framework of **machine learning** and **deep learning**. It is **a free** and **open-source** library developed by **Google Brain Team**. It is entirely based on Python programming language and use for numerical computation and data flow, which makes machine learning faster and easier.

TensorFlow can train and run the deep neural networks for image recognition, handwritten digit classification, recurrent neural network, **word embedding**, **natural language processing**, video detection, and many more. TensorFlow is run on multiple **CPU**s or **GPU**s and mobile operating systems.

The word TensorFlow is made by two words, i.e., Tensor and Flow

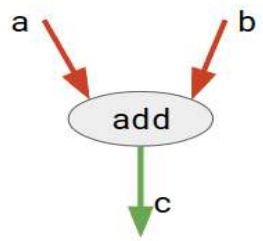
1. **Tensor** is a multidimensional array
2. **Flow** is used to define the flow of data in operation.

TensorFlow is used to define the flow of data in operation on a multidimensional array or Tensor.

TensorFlow is basically a software library for numerical computation using **data flow graphs** where:

- **nodes** in the graph represent mathematical operations.
- **edges** in the graph represent the multidimensional data arrays (called **tensors**) communicated between them. (Please note that **tensor** is the central unit of data in TensorFlow).

Consider the diagram given below:



Here, **add** is a node which represents addition operation. **a** and **b** are input tensors and **c** is the resultant tensor. This flexible architecture allows you to deploy computation to one or more CPUs or GPUs in a desktop, server, or mobile device with a single API!

The **MNIST database** (Modified National Institute of Standards and Technology database) is a large database of handwritten digits that is commonly used for training various image processing systems.

CODE:

```
import numpy as np

import matplotlib.pyplot as plt

import tensorflow

from tensorflow import keras

from tensorflow.keras import Sequential

from tensorflow.keras.layers import Flatten,Dense

(x_train,y_train), (x_test, y_test)=keras.datasets.mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```
print(x_train.shape)
print(y_train.shape)
print(x_test.shape)
print(y_test.shape)
```

```
(60000, 28, 28)
(60000,)
(10000, 28, 28)
(10000,)
```

x_train

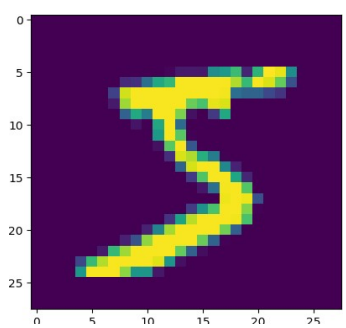
```
array([[[0, 0, 0, ..., 0, 0, 0],
 [0, 0, 0, ..., 0, 0, 0],
 [0, 0, 0, ..., 0, 0, 0],
 ...,
 [0, 0, 0, ..., 0, 0, 0],
 ...,
 [0, 0, 0, ..., 0, 0, 0],
 [0, 0, 0, ..., 0, 0, 0],
 [0, 0, 0, ..., 0, 0, 0]],

 [[0, 0, 0, ..., 0, 0, 0],
 [0, 0, 0, ..., 0, 0, 0],
 [0, 0, 0, ..., 0, 0, 0],

...,
```

```
[[0, 0, 0, ..., 0, 0, 0],
[0, 0, 0, ..., 0, 0, 0],
[0, 0, 0, ..., 0, 0, 0],
...,
[0, 0, 0, ..., 0, 0, 0],
[0, 0, 0, ..., 0, 0, 0],
[0, 0, 0, ..., 0, 0, 0]],

[[0, 0, 0, ..., 0, 0, 0],
[0, 0, 0, ..., 0, 0, 0],
[0, 0, 0, ..., 0, 0, 0],
...,
[0, 0, 0, ..., 0, 0, 0],
[0, 0, 0, ..., 0, 0, 0],
[0, 0, 0, ..., 0, 0, 0]]], dtype=uint8)
```

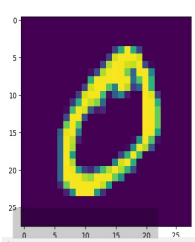
x_train[0]

ndarray (28, 28) show data

plt.imshow(x_train[0])



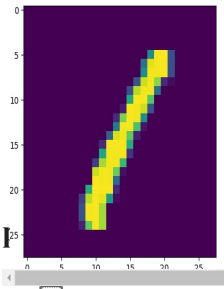
plt.imshow(x_train[1])



print(y_train[1])

0

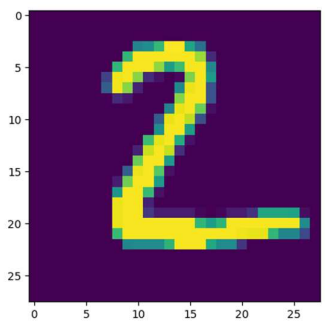
plt.imshow(x_train[3])



1

plt.imshow(x_test[1])
print(y_test[1])

2



x_train= x_train/255
x_test=x_test/255

x_train[0]

```
array([[0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        ],

       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.01176471, 0.07058824, 0.07058824,
       0.07058824, 0.49411765, 0.53333333, 0.68627451, 0.10196078,
       0.65098039, 1.        , 0.96862745, 0.49803922, 0.        ,
       0.        , 0.        , 0.        ],
      [0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.11764706, 0.14117647,
,
…….
       0.        , 0.04313725, 0.74509804, 0.99215686, 0.2745098 ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        ],
```

```
     [0.        , 0.        , 0.        , 0.        , 0.        ,
      0.        , 0.        , 0.        , 0.        , 0.        ,
      0.        , 0.        , 0.1372549 , 0.94509804, 0.88235294,
,.............
      0.        , 0.        , 0.        , 0.        , 0.        ,
      0.        , 0.        , 0.        , 0.        , 0.        ,
      0.        , 0.        , 0.        ],
     [0.        , 0.        , 0.        , 0.        , 0.53333333,
      0.99215686, 0.99215686, 0.99215686, 0.83137255, 0.52941176,
      0.51764706, 0.0627451 , 0.        , 0.        , 0.        ,
……  [0.        , 0.        , 0.        , 0.        , 0.        ,
      0.        , 0.        , 0.        , 0.        , 0.        ,
      0.        , 0.        , 0.        , 0.        , 0.        ,
      0.        , 0.        , 0.        , 0.        , 0.        ,
      0.        , 0.        , 0.        , 0.        , 0.        ,
      0.        , 0.        , 0.        ]])
```

```
model=Sequential()
model.add(Flatten(input_shape=(28,28)))
model.add(Dense(128,activation='relu'))
model.add(Dense(10,activation='softmax'))
```

```
model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 10)	1,290

Total params: 101,770 (397.54 KB)
Trainable params: 101,770 (397.54 KB)
Non-trainable params: 0 (0.00 B)

model.compile(optimizer='Adam',loss='sparse_categorical_crossentropy', metrics=['accuracy'])

history=model.fit(x_train,y_train,batch_size=64,epochs=10,verbose=1,validation_split=0.2)

```
Epoch 1/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.8454 - loss: 0.5525 -
val_accuracy: 0.9524 - val_loss: 0.1738
Epoch 2/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9550 - loss: 0.1550 -
val_accuracy: 0.9635 - val_loss: 0.1250
Epoch 3/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - accuracy: 0.9704 - loss: 0.1042 -
val_accuracy: 0.9655 - val_loss: 0.1142
Epoch 4/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9769 - loss: 0.0804 -
val_accuracy: 0.9701 - val_loss: 0.0992
Epoch 5/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.9820 - loss: 0.0632 -
val_accuracy: 0.9733 - val_loss: 0.0896
Epoch 6/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.9852 - loss: 0.0493 -
val_accuracy: 0.9724 - val_loss: 0.0903
Epoch 7/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9896 - loss: 0.0385 -
val_accuracy: 0.9724 - val_loss: 0.0951
Epoch 8/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.9908 - loss: 0.0321 -
val_accuracy: 0.9768 - val_loss: 0.0803
Epoch 9/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6s 6ms/step - accuracy: 0.9921 - loss: 0.0268 -
val_accuracy: 0.9740 - val_loss: 0.0846
Epoch 10/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9943 - loss: 0.0216 -
val_accuracy: 0.9767 - val_loss: 0.0836
```

model.evaluate(x_test,y_test)

```
313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9758 - loss: 0.0841

[0.07657930254936218, 0.9779000282287598]
```

model.predict(x_test)

```
313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step

array([[2.0060583e-08, 6.7596369e-09, 6.9048809e-07, ..., 9.9997061e-01,
        5.0159355e-07, 1.8256155e-06],
       [8.1633456e-09, 5.8597493e-06, 9.9999386e-01, ..., 2.0438967e-12,
        5.4751066e-08, 1.1290362e-11],
```

[4.2228148e-05, 9.9403298e-01, 3.4859095e-04, ..., 1.5076582e-03,
3.7968184e-03, 3.5331021e-05],
...,
[3.9407464e-09, 7.5120888e-13, 1.8149497e-09, ..., 5.7686836e-15,
1.4641963e-11, 1.3995148e-14]], dtype=float32)

np.argmax(model.predict(x_test), axis=-1)

313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step

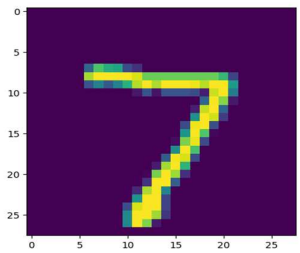
array([7, 2, 1, ..., 4, 5, 6])

np.argmax(model.predict(x_test), axis=-1).shape

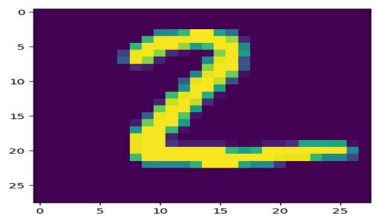
313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step

(10000,)

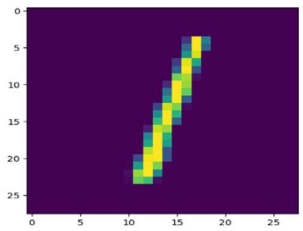
plt.imshow(x_test[0])



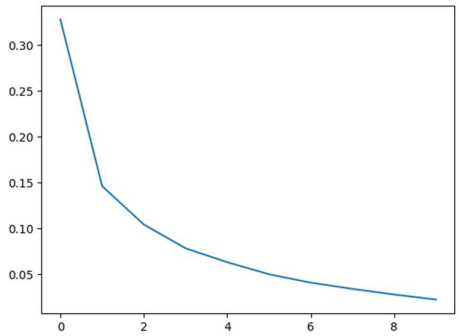
plt.imshow(x_test[1])



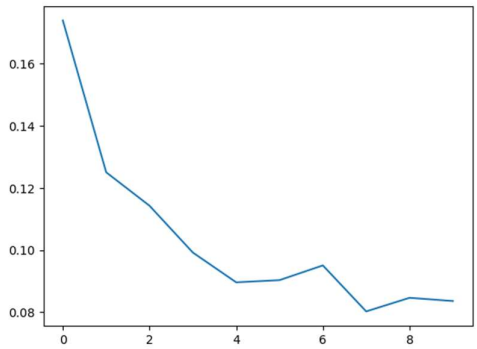
plt.imshow(x_test[2])



plt.plot(history.history['loss'])

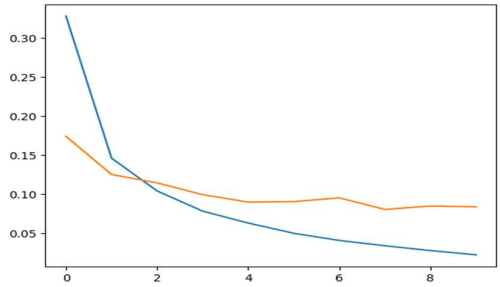


plt.plot(history.history['val_loss'])



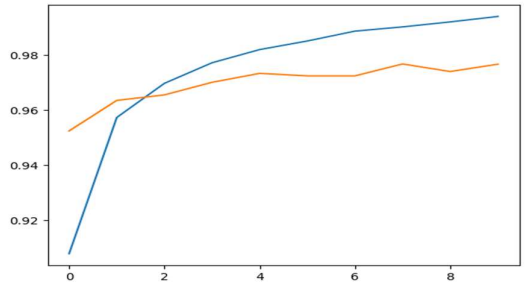
plt.plot(history.history['loss'])

plt.plot(history.history['val_loss'])



plt.plot(history.history['accuracy'])

plt.plot(history.history['val_accuracy'])



EXPERIMENT – 2

AIM: Design a Neural Network for classifying Movie Reviews (Binary Classification) using IMDB dataset.

Two-class classification, or binary classification, may be the most widely applied kind of machine learning problem. In this example, we will learn to classify movie reviews into "positive" reviews and "negative" reviews, just based on the text content of the reviews.

The IMDB dataset: Just like the MNIST dataset, the IMDB dataset comes packaged with Keras. It has already been preprocessed: the reviews (sequences of words) have been turned into sequences of integers, where each integer stands for a specific word in a dictionary.

The following code will load the dataset (when you run it for the first time, about 80MB of data will be downloaded to your machine):

```
from keras.datasets import imdb

(train_data, train_labels), (test_data, test_labels) =
imdb.load_data(num_words=10000)
```

The argument num_words=10000 means that we will only keep the top 10,000 most frequently occurring words in the training data. Rare words will be discarded. This allows us to work with vector data of manageable size.

The variables train_data and test_data are lists of reviews, each review being a list of word indices (encoding a sequence of words). train_labels and test_labels are lists of 0s and 1s, where 0 stands for "negative" and 1 stands for "positive":

train_data[0]

```
[1,
 14,
 22,
 16,
 43,
 530,
 973,
 1622,
 1385,
 65,
 458,
 4468,
 66,
```

train_labels[0]

```
1
```

 # Preparing_the_data

```
import numpy as np

def Vectorize_sequences(sequences,dimension=10000):
    results=np.zeros((len(sequences),dimension))
    for i,sequence in enumerate(sequences):
        results[i,sequence]=1
    return results
    Vectorize_sequences(train_data)
```

```
# Our vectorized training data
x_train = vectorize_sequences(train_data)

# Our vectorized test data
x_test = vectorize_sequences(test_data)
```

Here's what our samples look like now:

```
x_train[0]
```

array([0., 1., 1., ..., 0., 0., 0.])

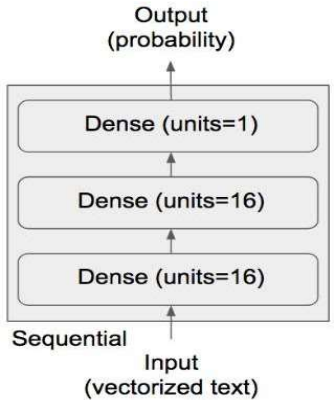
We should also vectorize our labels, which is straightforward:

```
# Our vectorized labels

y_train= np.asarray(train_labels).astype('float32')
y_test = np.asarray(test_labels).astype('float32')
```

Now our data is ready to be fed into a neural network.

 # Building our network

output = relu(dot(W, input) + b)



The Keras implementation Code:

```
from keras import models
from keras import layers

model=models.Sequential()
model.add(layers.Dense(16,activation='relu',input_shape=(10000,)))
model.add(layers.Dense(16,activation='relu'))
model.add(layers.Dense(1,activation='sigmoid'))
```

Lastly, we need to pick a loss function and an optimizer. Since we are facing a binary classification problem and the output of our network is a probability (we end our network with a single-unit layer with a sigmoid activation), is it best to use the binary_crossentropy loss. It isn't the only viable choice: you could use, for instance, mean_squared_error. But crossentropy is usually the best choice when you are dealing with models that output probabilities. Crossentropy is a quantity from the field of Information Theory, that measures the "distance" between probability distributions, or in our case, between the ground-truth distribution and our predictions.

Here's the step where we configure our model with the rmsprop optimizer and the binary_crossentropy loss function. Note that we will also monitor accuracy during training

```
model.compile(optimizer='rmsprop',loss='binary_crossentropy',metrics=['accuracy'])
```

 # Validating our approach

In order to monitor during training the accuracy of the model on data that it has never seen before, we will create a "validation set" by setting apart 10,000 samples from the original training data:

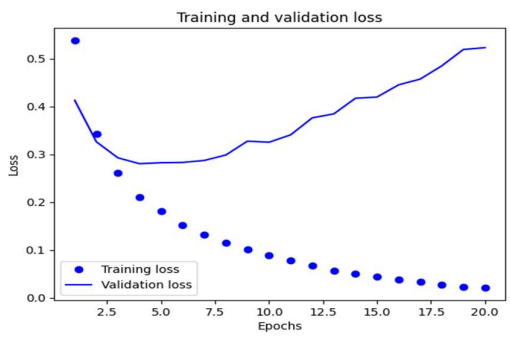
```
x_val= x_train[:10000]
partial_x_train=x_train[10000:]
y_val=y_train[:10000]
partial_y_train=y_train[10000:]
```

We will now train our model for 20 epochs (20 iterations over all samples in the x_train and y_train tensors), in mini-batches of 512 samples. At this same time we will monitor loss and accuracy on the 10,000 samples that we set apart. This is done by passing the validation data as the validation_data argument:

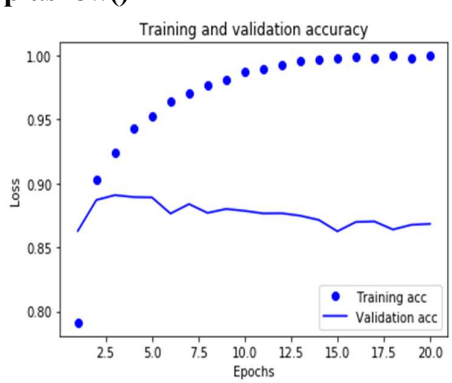
history=model.fit(partial_x_train,partial_y_train,epochs=20,batch_size=512,validation_data=(x_val,y_val))

```
Epoch 1/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 3s 61ms/step - accuracy: 0.7078 - loss: 0.6072 - val_accuracy:
0.8656 - val_loss: 0.4126
Epoch 2/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 32ms/step - accuracy: 0.8906 - loss: 0.3600 - val_accuracy:
0.8804 - val_loss: 0.3260
Epoch 3/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 31ms/step - accuracy: 0.9142 - loss: 0.2660 - val_accuracy:
0.8873 - val_loss: 0.2925
Epoch 4/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 32ms/step - accuracy: 0.9280 - loss: 0.2145 - val_accuracy:
0.8897 - val_loss: 0.2803
Epoch 5/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 33ms/step - accuracy: 0.9439 - loss: 0.1763 - val_accuracy:
0.8870 - val_loss: 0.2823
Epoch 6/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 34ms/step - accuracy: 0.9525 - loss: 0.1536 - val_accuracy:
0.8875 - val_loss: 0.2829
Epoch 7/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 33ms/step - accuracy: 0.9608 - loss: 0.1304 - val_accuracy:
0.8860 - val_loss: 0.2870
Epoch 8/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.9686 - loss: 0.1100 - val_accuracy:
0.8862 - val_loss: 0.2985
Epoch 9/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 2s 51ms/step - accuracy: 0.9722 - loss: 0.1000 - val_accuracy:
0.8812 - val_loss: 0.3274
Epoch 10/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 2s 33ms/step - accuracy: 0.9763 - loss: 0.0865 - val_accuracy:
0.8825 - val_loss: 0.3251
Epoch 11/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 31ms/step - accuracy: 0.9821 - loss: 0.0759 - val_accuracy:
0.8805 - val_loss: 0.3405
Epoch 12/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 32ms/step - accuracy: 0.9869 - loss: 0.0603 - val_accuracy:
0.8712 - val_loss: 0.3761
Epoch 13/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 33ms/step - accuracy: 0.9898 - loss: 0.0521 - val_accuracy:
0.8779 - val_loss: 0.3844
Epoch 14/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 34ms/step - accuracy: 0.9910 - loss: 0.0464 - val_accuracy:
0.8744 - val_loss: 0.4170
Epoch 15/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 32ms/step - accuracy: 0.9914 - loss: 0.0423 - val_accuracy:
0.8729 - val_loss: 0.4196
Epoch 16/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 31ms/step - accuracy: 0.9943 - loss: 0.0352 - val_accuracy:
0.8743 - val_loss: 0.4450
Epoch 17/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 33ms/step - accuracy: 0.9942 - loss: 0.0303 - val_accuracy:
0.8739 - val_loss: 0.4571
Epoch 20/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 2s 31ms/step - accuracy: 0.9986 - loss: 0.0195 - val_accuracy:
0.8704 - val_loss: 0.5228
```

```
import matplotlib.pyplot as plt

history_dict=history.history
loss_values=history_dict['loss']
val_loss_values=history_dict['val_loss']
epochs=range(1,len(loss_values)+1)


plt.plot(epochs,loss_values,'bo',label='Training loss')
plt.plot(epochs,val_loss_values,'b',label='Validation loss')
plt.title('Training and validation loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')

plt.legend()
plt.show()
```




```
plt.clf()          # clear figure
acc_values = history_dict['acc']
val_acc_values=history_dict['val_acc']
plt.plot(epochs, acc, 'bo', label='Training acc')
plt.plot(epochs, val_acc, 'b', label='Validation acc')
plt.title('Training and validation accuracy')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```



The dots are the training loss and accuracy, while the solid lines are the validation loss and accuracy. Note that your own results may vary slightly due to a different random initialization of your network.

Let's train a new network from scratch for four epochs, then evaluate it on our test data:

```
model = models.Sequential()
model.add(layers.Dense(16, activation='relu', input_shape=(10000,)))
model.add(layers.Dense(16, activation='relu'))
model.add(layers.Dense(1,activation='sigmoid'))

model.compile(optimizer='rmsprop', loss='binary_crossentropy', metrics=['accuracy'])

model.fit(x_train, y_train, epochs=4, batch_size=512)
results = model.evaluate(x_test, y_test)
```

```
25000/25000 [==============================] - 1s - loss: 0.4738 - acc: 0.8044
Epoch 2/4
25000/25000 [==============================] - 1s - loss: 0.2660 - acc: 0.9076
Epoch 3/4
25000/25000 [==============================] - 1s - loss: 0.2028 - acc: 0.9277
Epoch 4/4
25000/25000 [==============================] - 1s - loss: 0.1700 - acc: 0.9397
24544/25000 [============================>.] - ETA: 0s
```

```
results
```

[0.29184698499679568, 0.88495999999999997]

Our fairly naive approach achieves an accuracy of 88%. With state-of-the-art approaches, one should be able to get close to 95%.

#Using a trained network to generate predictions on new data

After having trained a network, you will want to use it in a practical setting. You can generate the likelihood of reviews being positive by using the predict method:

model.predict(x_test)

array([[0.91966152],

[0.86563045],

[0.99936908],

...,

[0.45731062],

[0.0038014],

[0.79525089]], dtype=float32)

Conclusion: As you can see, the network is very confident for some samples (0.99 or more, or 0.01 or less) but less confident for others (0.6, 0.4).

EXPERIMENT – 3

AIM: Design a Neural Network for classifying News wires (Multi class Classification) using Reuters Dataset.

"Classifying newswires" using multiclass classification means using a machine learning model to categorize news articles into multiple distinct categories (like "politics", "business", "sports", "technology") based on their content, where each news article can only belong to one of these categories, making it a multi-class problem as there are more than two possible classifications.

Example of classifying newswires:

- **Input:** A news article about a company's stock price fluctuation.
- **Features:** Keywords like "stock", "company name", "earnings", "share price", "market"
- **Output:** The model would classify the article as belonging to the "Business" category.

Multiclass classification is a type of supervised learning where the goal is to categorize instances (newswires, in this case) into one of three or more classes. For the Reuters dataset, each newswire is associated with a specific topic, such as **economy**, **sports**, **technology**, etc.

The Reuters Dataset

- **Description**: The Reuters dataset contains thousands of short newswires and their associated topic labels. It is included as part of the **Keras datasets module**.
- **Number of Classes**: There are 46 distinct topics (labels).
- **Input Data**: Each newswire is a sequence of words (text).
- **Output Labels**: Integer indices representing the topic classes.

<u>Keras Code Implementation:</u>

```
import keras
keras.__version__
from keras.datasets import reuters
(train_data, train_labels), (test_data, test_labels) = reuters.load_data(num_words=10000)

len(train_data)
```

8982

```
len(test_data)
```

2246

```
train_data[10]
```

```
[ 1,
  245,
  273,
  207,
  156,
  53,
  74,
  160,
  26,
  14,
```

Here's how you can decode it back to words

```
word_index = reuters.get_word_index()
reverse_word_index = dict([(value, key) for (key, value) in word_index.items()])

# Note that our indices were offset by 3
# because 0, 1 and 2 are reserved indices for "padding", "start of sequence", and "unknown".

decoded_newswire = ' '.join([reverse_word_index.get(i - 3, '?') for i in train_data[0]])

decoded_newswire
--→'? ? ? said as a result of its december acquisition of space co it expects
earnings per share in 1987 of 1 15 to 1 30 dlrs per share up from 70 cts in 1986
the company said pretax net should rise to nine to 10 mln dlrs from six mln
dlrs in 1986 and rental operation revenues to 19 to 22 mln dlrs from 12 5 mln
dlrs it said cash flow per share this year should be 2 50 to three dlrs reuter
3'
```

The label associated with an example is an integer between 0 and 45: a topic index.

```
train_labels[10]
--→ 3
```

#Preparing the data

```
import numpy as np

def vectorize_sequences(sequences, dimension=10000):
    results = np.zeros((len(sequences), dimension))
    for i, sequence in enumerate(sequences):
        results[i, sequence] = 1.
    return results


# Vectorize the training and validation data
x_train = vectorize_sequences(train_data)

# Our vectorized test data
x_test = vectorize_sequences(test_data)

x_train,x_train.shape,x_train.ndim
```

```
--→
(array([[0., 1., 1., ..., 0., 0., 0.],
        [0., 1., 1., ..., 0., 0., 0.],
        [0., 1., 1., ..., 0., 0., 0.],
        ...,
        [0., 1., 1., ..., 0., 0., 0.],
        [0., 1., 1., ..., 0., 0., 0.],
        [0., 1., 1., ..., 0., 0., 0.]]),
 (8982, 10000),
 2)
```

```
data=[1,2,4,9,0]
x= vectorize_sequences(data)
x,x.shape,x.ndim
```

```
--→
(array([[0., 1., 0., ..., 0., 0., 0.],
        [0., 0., 1., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.],
        [1., 0., 0., ..., 0., 0., 0.]]),
 (5, 10000),
 2)
```

To vectorize the labels, there are two possibilities: we could just cast the label list as an integer tensor, or we could use a "one-hot" encoding. One-hot encoding is a widely used format for categorical data, also called "categorical encoding".In our case, one-hot encoding of our labels consists in embedding each label as an all-zero vector with a 1 in the place of the label index, e.g.:

```
def to_one_hot(labels, dimension=46):
    results = np.zeros((len(labels), dimension))
    for i, label in enumerate(labels):
        results[i, label] = 1.
    return results


# Our vectorized training labels
one_hot_train_labels = to_one_hot(train_labels)


# Our vectorized test labels
one_hot_test_labels = to_one_hot(test_labels)
one_hot_train_labels,one_hot_train_labels.shape,one_hot_train_labels.ndim
```

```
--→ (array([[0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.],
        ...,
```

```
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.]]),
 (8982, 46),
 2)
```

```
from keras.utils.np_utils import to_categorical
one_hot_train_labels = to_categorical(train_labels)
one_hot_test_labels = to_categorical(test_labels)
```

 # Building our network

There is however a new constraint here: the number of output classes has gone from 2 to 46, i.e. the dimensionality of the output space is much larger. permanently dropping relevant information.

For this reason we will use larger layers. Let's go with 64 units:

```
from keras import models
from keras import layers

model=models.Sequential()
model.add(layers.Dense(64, activation='relu', input_shape=(10000,)))
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(46, activation='softmax'))
```

There are two other things you should note about this architecture:

- We are ending the network with a Dense layer of size 46. This means that for each input sample, our network will output a 46-dimensional vector. Each entry in this vector (each dimension) will encode a different output class.

- The last layer uses a softmax activation. It means that the network will output a *probability distribution* over the 46 different output classes, i.e. for every input sample, the network will produce a 46-dimensional output vector where output[i] is the probability that the sample belongs to class i. The 46 scores will sum to 1.

The best loss function to use in this case is categorical_crossentropy. It measures the distance between two probability distributions: in our case, between the probability distribution output by our network, and the true distribution of the labels. By minimizing the distance between these two distributions, we train our network to output something as close as possible to the true labels.

```
model.compile(optimizer='rmsprop',loss='categorical_crossentropy', metrics=['accuracy'])
```

 # Validating our approach

Let's set apart 1,000 samples in our training data to use as a validation set:

```
x_val = x_train[:1000]

partial_x_train = x_train[1000:]

y_val = one_hot_train_labels[:1000]

partial_y_train = one_hot_train_labels[1000:]
```

Now let's train our network for 20 epochs:

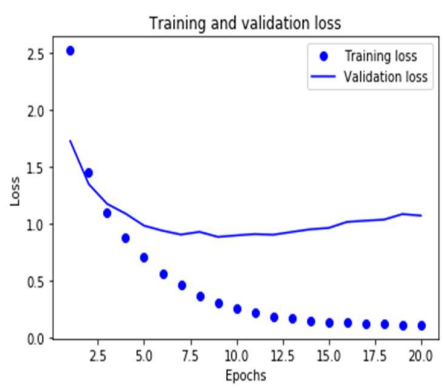
```
history = model.fit(partial_x_train,   partial_y_train, epochs=20, batch_size=512,
validation_data=(x_val, y_val))
```

```
Epoch 1/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 65ms/step - accuracy: 0.4126 - loss: 3.1747 - val_accuracy:
0.6400 - val_loss: 1.7932
Epoch 2/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.6799 - loss: 1.5761 - val_accuracy:
0.6970 - val_loss: 1.3661
Epoch 3/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - accuracy: 0.7424 - loss: 1.1933 - val_accuracy:
0.7310 - val_loss: 1.2008
Epoch 4/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - accuracy: 0.7891 - loss: 0.9738 - val_accuracy:
0.7520 - val_loss: 1.1137
Epoch 5/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - accuracy: 0.8232 - loss: 0.8135 - val_accuracy:
0.7780 - val_loss: 1.0144
Epoch 6/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.8516 - loss: 0.6852 - val_accuracy:
0.8030 - val_loss: 0.9650
Epoch 7/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.8871 - loss: 0.5504 - val_accuracy:
0.8020 - val_loss: 0.9217
Epoch 8/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.9088 - loss: 0.4498 - val_accuracy:
0.8010 - val_loss: 0.9050
Epoch 9/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 83ms/step - accuracy: 0.9223 - loss: 0.3715 - val_accuracy:
0.8130 - val_loss: 0.8858
Epoch 10/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 83ms/step - accuracy: 0.9334 - loss: 0.3278 - val_accuracy:
0.8080 - val_loss: 0.8970
Epoch 11/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 58ms/step - accuracy: 0.9388 - loss: 0.2917 - val_accuracy:
0.8080 - val_loss: 0.8866
Epoch 12/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.9495 - loss: 0.2363 - val_accuracy:
0.8120 - val_loss: 0.8915
Epoch 13/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.9528 - loss: 0.1956 - val_accuracy:
0.8030 - val_loss: 0.8964
```

----------1s 45ms/step - accuracy: 0.9549 - loss: 0.1723 - val_accuracy: 0.8110 - val_loss: 0.9315
Epoch 17/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - accuracy: 0.9551 - loss: 0.1519 - val_accuracy: 0.8090 - val_loss: 0.9301
Epoch 18/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - accuracy: 0.9618 - loss: 0.1285 - val_accuracy: 0.8030 - val_loss: 0.9521
Epoch 19/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.9601 - loss: 0.1348 - val_accuracy: 0.8090 - val_loss: 0.9429
Epoch 20/20
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.9623 - loss: 0.1279 - val_accuracy: 0.8020 - val_loss: 0.9960

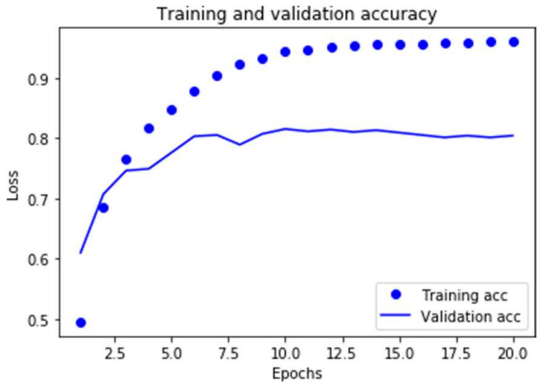
#Displaying its loss and accuracy curves:

```
import matplotlib.pyplot as plt

loss = history.history['loss']
val_loss = history.history['val_loss']

epochs = range(1, len(loss) + 1)

plt.plot(epochs, loss, 'bo', label='Training loss')
plt.plot(epochs, val_loss, 'b', label='Validation loss')
plt.title('Training and validation loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.show()
```




```
# Clear the figure

plt.clf()

# Access the correct history keys

acc = history.history.get('accuracy', [])

val_acc = history.history.get('val_accuracy', [])
```

```
# Define the number of epochs
epochs = range(1, len(acc) + 1)


# Plot training and validation accuracy
plt.plot(epochs, acc, 'bo', label='Training acc')
plt.plot(epochs, val_acc, 'b', label='Validation acc')
plt.title('Training and validation accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
# Display the plot
plt.show()
```



It seems that the network starts overfitting after 8 epochs. Let's train a new network from scratch for 8 epochs, then let's evaluate it on the test set:

```
model = models.Sequential()
model.add(layers.Dense(64, activation='relu', input_shape=(10000,)))
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(46, activation='softmax'))
model.compile(optimizer='rmsprop',loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(partial_x_train, partial_y_train, epochs=8, batch_size=512, validation_data=(x_val,
y_val))
results = model.evaluate(x_test, one_hot_test_labels)
```

Epoch 1/8

16/16 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 74ms/step - accuracy: 0.3553 - loss: 3.1928 - val_accuracy: 0.6210 - val_loss: 1.8288

Epoch 2/8

16/16 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - accuracy: 0.6561 - loss: 1.6461 - val_accuracy: 0.6860 - val_loss: 1.3899

Epoch 3/8

16/16 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - accuracy: 0.7411 - loss: 1.2058 - val_accuracy: 0.7380 - val_loss: 1.2231

Epoch 4/8

16/16 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - accuracy: 0.7936 - loss: 0.9919 - val_accuracy: 0.7630 - val_loss: 1.1063

--------Epoch 6/8

16/16 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 82ms/step - accuracy: 0.8581 - loss: 0.6837 - val_accuracy: 0.8000 - val_loss: 0.9624

Epoch 7/8

16/16 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 87ms/step - accuracy: 0.8780 - loss: 0.5762 - val_accuracy: 0.8030 - val_loss: 0.9340

Epoch 8/8

16/16 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 57ms/step - accuracy: 0.9035 - loss: 0.4524 - val_accuracy: 0.8110 - val_loss: 0.9141

71/71 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7928 - loss: 0.9518

results

➔ [0.9859249591827393, 0.7813891172409058]

Our approach reaches an accuracy of ~78%. With a balanced binary classification problem, the accuracy reached by a purely random classifier would be 50%, but in our case it is closer to 19%, so our results seem pretty good, at least when compared to a random baseline:

```
import copy

test_labels_copy = copy.copy(test_labels)
np.random.shuffle(test_labels_copy)
float(np.sum(np.array(test_labels) == np.array(test_labels_copy))) / len(test_labels)
```

➔ 0.18432769367764915

 # Generating predictions on new data

We can verify that the predict method of our model instance returns a probability distribution over all 46 topics. Let's generate topic predictions for all of the test data:

```
predictions = model.predict(x_test)
```

Each entry in predictions is a vector of length 46:

predictions[0].shape

-→ (46,)

The coefficients in this vector sum to 1:

np.sum(predictions[0])

→ 1.0000001

The largest entry is the predicted class, i.e. the class with the highest probability:

np.argmax(predictions[0])

way to handle the labels and the loss

```
y_train = np.array(train_labels)
y_test = np.array(test_labels)
y_train,y_train.shape,y_train.ndim
```

-→ (array([3, 4, 3, ..., 25, 3, 25]), (8982,), 1)

The only thing it would change is the choice of the loss function. Our previous loss, categorical_crossentropy, expects the labels to follow a categorical encoding. With integer labels, we should use sparse_categorical_crossentropy:

```
model.compile(optimizer='rmsprop', loss='sparse_categorical_crossentropy', metrics=['acc'])
```

Importance of having sufficiently large intermediate layers

We mentioned earlier that since our final outputs were 46-dimensional, we should avoid intermediate layers with much less than 46 hidden units. Now let's try to see what happens when we introduce an information bottleneck by having intermediate layers significantly less than 46-dimensional, e.g. 4- dimensional.

```
model = models.Sequential()
model.add(layers.Dense(64, activation='relu', input_shape=(10000,)))
model.add(layers.Dense(4, activation='relu'))
model.add(layers.Dense(46, activation='softmax'))

model.compile(optimizer='rmsprop', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(partial_x_train, partial_y_train, epochs=20,batch_size=128,
          validation_data=(x_val, y_val))
```

Epoch 1/20
63/63 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - accuracy: 0.2283 - loss: 3.3369 - val_accuracy: 0.5200 - val_loss: 2.2475

```
Epoch 2/20
63/63 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.5462 - loss: 2.0714 -
val_accuracy: 0.5710 - val_loss: 1.7970

Epoch 3/20
63/63 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - accuracy: 0.5827 - loss: 1.7319 -
val_accuracy: 0.6030 - val_loss: 1.6658

Epoch 4/20
63/63 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - accuracy: 0.6149 - loss: 1.5636 -
val_accuracy: 0.6060 - val_loss: 1.6233

Epoch 5/20
63/63 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - accuracy: 0.6162 - loss: 1.4792 -
val_accuracy: 0.6050 - val_loss: 1.5901------------
--------------
```

```
Epoch 13/20
63/63 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - accuracy: 0.7344 - loss: 0.9895 -
val_accuracy: 0.6820 - val_loss: 1.5341
Epoch 14/20
63/63 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - accuracy: 0.7592 - loss: 0.9564 -
val_accuracy: 0.6870 - val_loss: 1.5454
Epoch 15/20
63/63 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - accuracy: 0.7699 - loss: 0.9195 -
val_accuracy: 0.6990 - val_loss: 1.5890

Epoch 16/20
63/63 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 28ms/step - accuracy: 0.7893 - loss: 0.8777 -
val_accuracy: 0.7020 - val_loss: 1.5625
Epoch 17/20
63/63 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 24ms/step - accuracy: 0.8017 - loss: 0.8008 -
val_accuracy: 0.7080 - val_loss: 1.5583
Epoch 18/20
63/63 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 15ms/step - accuracy: 0.8159 - loss: 0.7354 -
val_accuracy: 0.7060 - val_loss: 1.5765
Epoch 19/20
63/63 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - accuracy: 0.8151 - loss: 0.7181 -
val_accuracy: 0.7100 - val_loss: 1.5808
Epoch 20/20
63/63 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - accuracy: 0.8290 - loss: 0.6515 -
val_accuracy: 0.7150 - val_loss: 1.6305
<keras.src.callbacks.history.History at 0x7b58bd0777f0>
```

Conclusion:

Our network now seems to peak at ~71% test accuracy, a 8% absolute drop. This drop is mostly due to the fact that we are now trying to compress a lot of information (enough information to recover the separation hyperplanes of 46 classes) into an intermediate space that is too low-dimensional. The network is able to cram *most* of the necessary information into these 8-dimensional representations, but not all of it.

EXPERIMENT – 4

AIM: Design a Neural Network for predicting House Prices using Boston Housing Price Dataset.

Another common type of machine learning problem is "regression", which consists of predicting a continuous value instead of a discrete label. For instance, predicting the temperature tomorrow, given meteorological data, or predicting the time that a software project will take to complete, given its specifications.

The Boston Housing Price dataset

We will be attempting to predict the median price of homes in a given Boston suburb in the mid- 1970s, given a few data points about the suburb at the time, such as the crime rate, the local property tax rate, etc.

The dataset we will be using has another interesting difference from our two previous examples: it has very few data points, only 506 in total, split between 404 training samples and 102 test samples, and each "feature" in the input data (e.g. the crime rate is a feature) has a different scale. For instance some values are proportions, which take a values between 0 and 1, others take values between 1 and 12, others between 0 and 100...

 # Keras implementaion Code:

```
import tensorflow as tf

from tensorflow.keras.datasets import boston_housing

from sklearn import preprocessing

(train_x, train_y), (test_x, test_y) = boston_housing.load_data()



print ("Train shape: ", train_x.shape)

print ("Test shape: ", test_x.shape)

print("Actual train output: ", train_y.shape)

print("Actual test output: ", test_y.shape)
```

---→

Train shape: (404, 13)

Test shape: (102, 13)

Actual train output: (404,)

Actual test output: (102,)

As you can see, we have 404 training samples and 102 test samples. The data comprises 13 features. The 13 features in the input data are as follow:

1. Per capita crime rate.
2. Proportion of residential land zoned for lots over 25,000 square feet.
3. Proportion of non-retail business acres per town.
4. Charles River dummy variable (= 1 if tract bounds river; 0 otherwise).
5. Nitric oxides concentration (parts per 10 million).
6. Average number of rooms per dwelling.
7. Proportion of owner-occupied units built prior to 1940.
8. Weighted distances to five Boston employment centres.
9. Index of accessibility to radial highways.
10. Full-value property-tax rate per $10,000.
11. Pupil-teacher ratio by town.
12. 1000 * (Bk - 0.63) ** 2 where Bk is the proportion of Black people by town.
13. % lower status of the population.

The targets are the median values of owner-occupied homes, in thousands of dollars:

train_x[0]

-→ array([1.23247, 0. , 8.14 , 0. , 0.538 , 6.142 ,

91.7 , 3.9769 , 4. , 307. , 21. , 396.9 ,

18.72])

train_y[0]

-→ 15.2

train_x=preprocessing.normalize(train_x)

test_x=preprocessing.normalize(test_x)

train_x[0]

--→array([0.0024119 , 0. , 0.01592969, 0. , 0.00105285,

0.01201967, 0.17945359, 0.00778265, 0.00782786, 0.6007879 ,

0.04109624, 0.77671895, 0.03663436])

train_x[1]

-→ array([4.07923050e-05, 1.54587284e-01, 3.80378407e-03, 0.00000000e+00,

7.77620881e-04, 1.42595058e-02, 2.94184285e-02, 1.17486336e-02,

3.74757051e-03, 6.52077269e-01, 2.75446433e-02, 7.40857215e-01,

5.82747215e-03])

train_y[0]

-→15.2

 # Building our network

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import *
def HousePricePredictionModel():
    model=Sequential()
    model.add(Dense(128,activation='relu',input_shape=(train_x[0].shape)))
    model.add(Dense(64,activation='relu'))
    model.add(Dense(32,activation='relu'))
    model.add(Dense(1))
    model.compile(optimizer='rmsprop',loss='mse',metrics=['mae'])
    return model
```

 # Validating our approach using K-fold validation

To evaluate our network while we keep adjusting its parameters (such as the number of epochs used for training), we could simply split the data into a training set and a validation set, as we were doing in our previous examples. However, because we have so few data points, the validation set would end up being very small (e.g. about 100 examples).

The best practice in such situations is to use K-fold cross-validation. It consists of splitting the available data into K partitions (typically K=4 or 5), then instantiating K identical models, and training each one on K-1 partitions while evaluating on the remaining partition. The validation score for the model used would then be the average of the K validation scores obtained.

In terms of code, this is straightforward:

```
import numpy as np
k=4
num_val_samples=len(train_x)
num_epochs=100
all_scores=[]

model=HousePricePredictionModel()
history=model.fit(x=train_x,y=train_y,epochs=num_epochs,batch_size=1,verbose=1,validation_data=(test_x,test_y))
```

```
Epoch 1/100
404/404 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - loss: 222.9861 - mae: 11.3155 -
val_loss: 71.9354 - val_mae: 5.9626

Epoch 2/100
404/404 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 82.4570 - mae: 6.2977 -
val_loss: 63.1096 - val_mae: 5.6932

Epoch 3/100
404/404 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 55.8918 - mae: 5.0503 -

Epoch 99/100
```

404/404 ━━━━━━━━━━━━━━━━━━━━ **3s 7ms/step - loss: 14.9275 - mae: 2.9780 - val_loss: 39.9865 - val_mae: 4.4459**

Epoch 100/100
404/404 ━━━━━━━━━━━━━━━━━━━━ **3s 2ms/step - loss: 14.9566 - mae: 2.8828 - val_loss: 33.0925 - val_mae: 3.8636**

 # Predicting with Some random input values

```
test_input=[[8.65407330e-05, 0.00000000e+00, 1.13392175e-02, 0.00000000e+00,
    1.12518247e-03, 1.31897603e-02, 7.53763011e-02, 1.30768051e-02,
    1.09241016e-02, 4.89399752e-01, 4.41333705e-02, 8.67155186e-01,
    1.75004108e-02]]
test_input = np.array(test_input)
print("Actual output: 21.1")
print("Predicted output: ",model.predict(test_input))
```

--→

Actual output: 21.1

1/1 ━━━━━━━━━━━━━━━━━━━━ **0s** 34ms/step

Predicted output: [[22.687279]]

```
test_input=[[0.0024119 , 0.        , 0.01592969, 0.        , 0.00105285,
    0.01201967, 0.17945359, 0.00778265, 0.00782786, 0.6007879 ,
    0.04109624, 0.77671895, 0.03663436]]
```

 # Convert test_input to a NumPy array

```
test_input = np.array(test_input)
print("Actual output: ",test_y[0])
print("Predicted output: ",model.predict(test_input))
```

---→

Actual output: 7.2

1/1 ━━━━━━━━━━━━━━━━━━━━ **0s** 99ms/step

Predicted output: [[17.433893]]

```
test_input=[[8.65407330e-05, 0.00000000e+00, 1.13392175e-02, 0.00000000e+00,
    1.12518247e-03, 1.31897603e-02, 7.53763011e-02, 1.30768051e-02,
    1.09241016e-02, 4.89399752e-01, 4.41333705e-02, 8.67155186e-01,
    1.75004108e-02]]
```

 # Convert test_input to a NumPy array

```
test_input = np.array(test_input)
print("Actual output: 21.1 ")
print("Predicted output: ",model.predict(test_input))
```

--→

```
Actual output: 21.1
1/1 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step
Predicted output:  [[22.687279]]
```

Conclusion:

- Regression is done using different loss functions from classification
- Mean Squared Error (MSE) is a commonly used loss function for regression.
- Similarly, evaluation metrics to be used for regression differ from those used for classification; naturally the concept of "accuracy" does not apply for regression.
- A common regression metric is Mean Absolute Error (MAE).

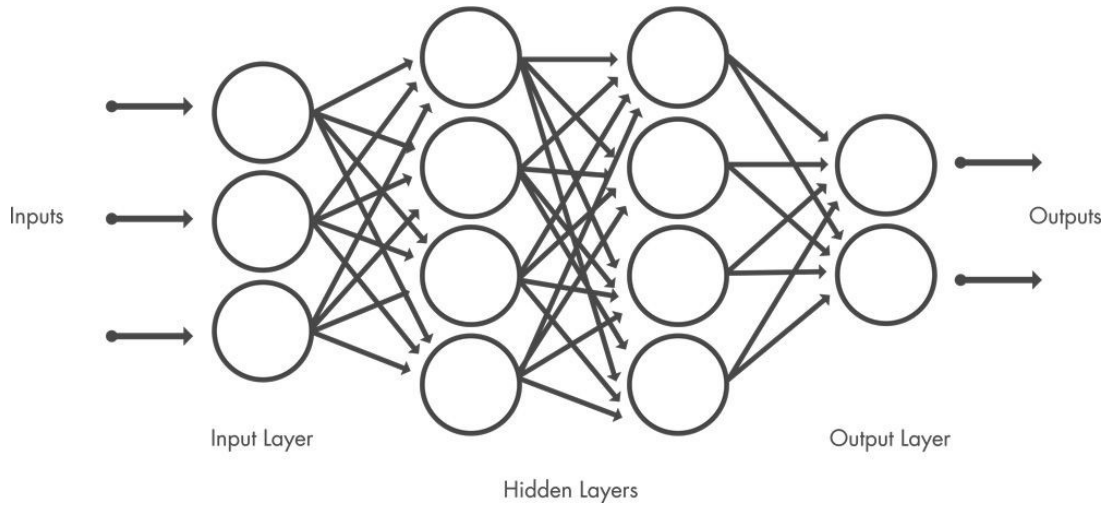
EXPERIMENT – 5

AIM: Build a Convolutional Neural Network for MNIST Hand written Digit classification.

A Convolutional Neural Network (CNN or ConvNet) is a network architecture for deep learning that learns directly from data.A CNN is composed of an input layer, an output layer, and many hidden layers in between.

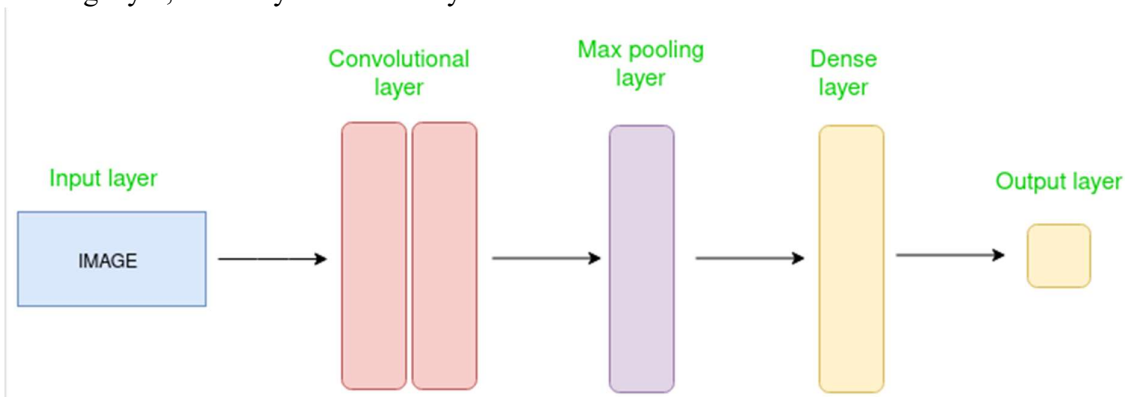
CNNs are particularly useful for finding patterns in images to recognize objects, classes and categories. They can also be quite effective for classifying audio, time series and signal data.

A convolutional neural network can have tens or hundreds of layers that each learn to detect different features of an image. Filters are applied to each training image at different resolutions, and the output of each convolved image is used as the input to the next layer. The filters can start as very simple features, such as brightness and edges, and increase in complexity to features that uniquely define the object



CNN Architecture

Convolutional Neural Network consists of multiple layers like the input layer, Convolutional layer, Pooling layer, and fully connected layers.



The Convolutional layer applies filters to the input image to extract features, the Pooling layer down samples the image to reduce computation, and the fully connected layer makes the final prediction. The network learns the optimal filters through backpropagation and gradient descent.

Keras Code Implemenation:

```
from numpy import unique,argmax
from tensorflow.keras.datasets.mnist import load_data
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Conv2D
from tensorflow.keras.layers import MaxPool2D
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Dropout
from tensorflow.keras.utils import plot_model
from matplotlib import pyplot as plt

import numpy as np
(x_train,y_train),(x_test,y_test)=load_data()

#loading the new datset
x_train=x_train.reshape((x_train.shape[0],x_train.shape[1],x_train.shape[2],1))

#reshaping the training and testing data
x_test=x_test.reshape((x_test.shape[0],x_test.shape[1],x_test.shape[2],1))

#normalizing the values of pixels of images
x_train=x_train.astype('float32')/255
x_test=x_test.astype('float32')/255

fig=plt.figure(figsize=(5,3))
for i in range(15):
    ax = fig.add_subplot(2, 10, i + 1, xticks=[], yticks=[])
    ax.imshow(np.squeeze(x_train[i]),cmap='gray')
    ax.set_title('Label:{}'.format(y_train[i]))
```

```
img_shape=x_train.shape[1:]
print(img_shape)
```

-→ (28, 28, 1)

 # defining the model

```
model=Sequential()
model.add(Conv2D(32,(3,3),activation='relu',kernel_initializer='he_uniform',input_shape=img_shape))
model.add(MaxPool2D((2,2)))
model.add(Flatten())
model.add(Dense(100,activation='relu',kernel_initializer='he_uniform'))
model.add(Dropout(0.5))
model.add(Dense(10,activation='softmax'))

model.summary()
```

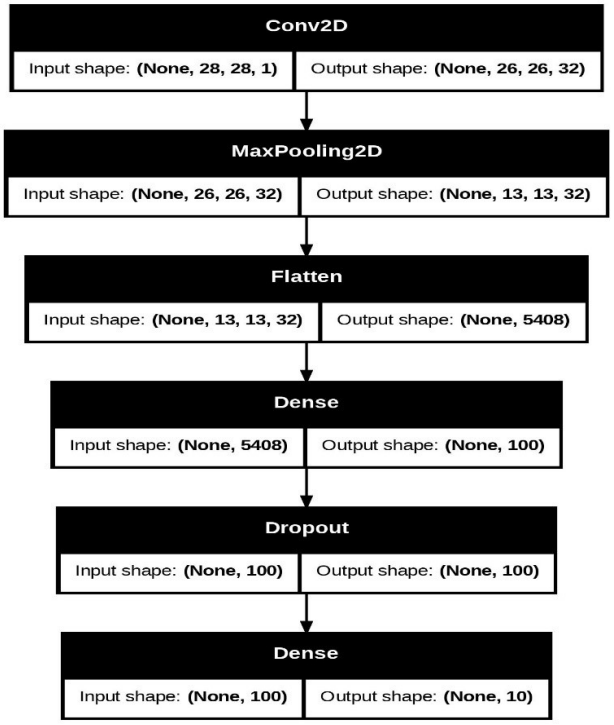
------→

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_1 (MaxPooling2D)	(None, 13, 13, 32)	0
flatten_1 (Flatten)	(None, 5408)	0
dense_2 (Dense)	(None, 100)	540,900
dropout_1 (Dropout)	(None, 100)	0
dense_3 (Dense)	(None, 10)	1,010

Total params: 542,230 (2.07 MB)
Trainable params: 542,230 (2.07 MB)
Non-trainable params: 0 (0.00 B)

```
plot_model(model, 'model.jpg',show_shapes=True)
```




```
model.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])
x=model.fit(x_train,y_train, epochs=10,batch_size=128, verbose=2, validation_split=0.1)
```

```
Epoch 1/10
422/422 - 26s - 61ms/step - accuracy: 0.8974 - loss: 0.3404 - val_accuracy: 0.9773 - val_loss: 0.0808
Epoch 2/10
422/422 - 41s - 97ms/step - accuracy: 0.9577 - loss: 0.1435 - val_accuracy: 0.9842 - val_loss: 0.0545
Epoch 3/10
422/422 - 41s - 97ms/step - accuracy: 0.9687 - loss: 0.1055 - val_accuracy: 0.9847 - val_loss: 0.0537
Epoch 4/10
422/422 - 41s - 97ms/step - accuracy: 0.9739 - loss: 0.0872 - val_accuracy: 0.9848 - val_loss: 0.0514
422/422 - 40s - 96ms/step - accuracy: 0.9831 - loss: 0.0531 - val_accuracy: 0.9865 - val_loss: 0.0486
Epoch 5/10
422/422 - 42s - 99ms/step - accuracy: 0.9770 - loss: 0.0742 - val_accuracy: 0.9870 - val_loss: 0.0467
Epoch 6/10
422/422 - 39s - 93ms/step - accuracy: 0.9793 - loss: 0.0637 - val_accuracy: 0.9877 - val_loss: 0.0444
Epoch 7/10
422/422 - 40s - 95ms/step - accuracy: 0.9827 - loss: 0.0564 - val_accuracy: 0.9863 - val_loss: 0.0483
```

Epoch 8/10

422/422 - 40s - 96ms/step - accuracy: 0.9831 - loss: 0.0531 - val_accuracy: 0.9865 - val_loss: 0.0486

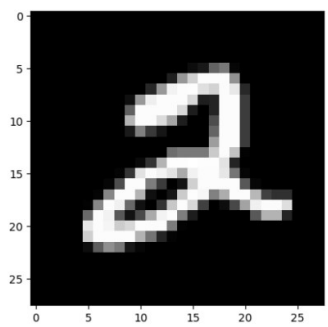
Epoch 9/10

422/422 - 42s - 99ms/step - accuracy: 0.9843 - loss: 0.0483 - val_accuracy: 0.9880 - val_loss: 0.0477

Epoch 10/10

422/422 - 42s - 98ms/step - accuracy: 0.9855 - loss: 0.0449 - val_accuracy: 0.9875 - val_loss: 0.0489

```
loss,accuracy=model.evaluate(x_test,y_test, verbose=0)
print(f'Accuracy: {accuracy*100}')

img=x_train[5]
plt.imshow(np.squeeze(img),cmap='gray') # Changed 'image' to 'img' and 'cmp' to 'cmap'
plt.show()
```




```
# Replace 'image' with 'img' to use the image data loaded previously

img = img.reshape(1, img.shape[0], img.shape[1], img.shape[2])
p = model.predict([img])
print('Predicted :{}'.format(argmax(p)))
```

1/1 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 120ms/step

Predicted :2

/usr/local/lib/python3.11/dist-packages/keras/src/models/functional.py:237: UserWarning: The structure of `inputs` doesn't match the expected structure.

Expected: keras_tensor

Received: inputs=('Tensor(shape=(1, 28, 28, 1))',)

  warnings.warn(msg)